

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04 – Informe Técnico Individual

Análisis de Rendimiento Paralelo (Unidad 4)
aplicado al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad: Unidad 4 – Cómputo Paralelo y Distribuido
Docente responsable: Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico: 2026–2027 PPA
Fecha: 8 de agosto de 2026

Datos del estudiante

Nombre completo: Freddy Vladimir Farinango Guandinango
Equipo de PE-U4: Equipo C (Farinango, Gaibor, Villamarín)
PFC de referencia: FUVV – Sistema de Control de Laboratorios Informáticos (SCLI)
Transformación foco: T2 – Agrupación y agregación
Repositorio individual: <https://github.com/ffarinangog2/ta-ind-04-Farinango>

Declaración de individualidad. Los datos experimentales de partida (medianas de tiempo de PE-U4) proceden legítimamente del trabajo grupal del Equipo C. El análisis, la interpretación, las ecuaciones resueltas, la figura, el diagrama y las conclusiones de este documento son de elaboración estrictamente individual. La transformación T2 fue coordinada con los demás integrantes del equipo como foco exclusivo de este informe, sin solaparse con los focos declarados por Jeremy Gaibor e Iván Villamarín.

Análisis de Rendimiento Paralelo de la Transformación T2 (Agrupación y Agregación) y su Integración en el Sistema de Control de Laboratorios Informáticos (SCLI)

Freddy Vladimir Farinango Guandinango
Ingeniería de Software, Universidad Técnica Estatal de Quevedo
ffarinangog2@uteq.edu.ec

Resumen—Este informe individual retoma las mediciones de la práctica grupal PE-U4 y analiza en profundidad la transformación T2 (agrupación y agregación por laboratorio) del pipeline de benchmarking pandas-PySpark. El speedup medido de T2 resulta inferior a la unidad en las tres configuraciones evaluadas ($N = 1, 2, 4$ executors), lo que invalida la aplicación directa de la métrica de Karp-Flatt y exige una explicación basada en el mecanismo de shuffle del motor distribuido. A partir de un modelo de costo marginal ajustado sobre las mediciones propias, se estima que, con la configuración de clúster usada en PE-U4 ($N = 4$), ningún volumen de datos finito rentabiliza el uso de Spark para esta operación, y se deriva el número mínimo de executors a partir del cual la rentabilidad vuelve a ser alcanzable. Con este diagnóstico se propone un punto de integración del procesamiento distribuido en la arquitectura del Sistema de Control de Laboratorios Informáticos (SCLI) y se argumenta por qué, en el estado actual del sistema, un motor distribuido no es la herramienta adecuada para este tipo de reporte.

Repositorio: <https://github.com/ffarinangog2/pe-u4-spark-equipo-c>

Commit: a5ffcd9 (fusión de la medición de escalado de T2 con $N = 1$ y $N = 2$, commit individual bbe19c8, sobre main)

Las mediciones de T2 con $N = 4$ executors, junto con las de las demás transformaciones, provienen de la entrega grupal original; las mediciones de T2 con $N = 1$ y $N = 2$ fueron ejecutadas individualmente por el autor de este informe, con idéntico protocolo, y añadidas al repositorio grupal en el commit declarado, dado que la guía de PE-U4 solo exigía el escalado completo para T3.

El foco individual declarado para este informe es **T2 (agrupación y agregación)**, coordinado previamente con los demás integrantes del Equipo C para evitar coincidencia de foco. El dominio de referencia es el PFC FUVV, *Sistema de Control de Laboratorios Informáticos* (SCLI), un sistema de reserva y monitoreo distribuido de laboratorios de la Facultad de Ciencias de la Computación.

I. INTRODUCCIÓN Y CONTEXTO

El objetivo de este informe es someter a análisis individual los resultados de speedup obtenidos por el Equipo C en la práctica GA-SUM-05 / PE-U4, contrastarlos cuantitativamente con la Ley de Amdahl [1] y, a partir del diagnóstico resultante, decidir y argumentar cómo integrar el procesamiento distribuido de datos en el Proyecto Fin de Curso (PFC) propio. A diferencia de PE-U4, que evaluó la capacidad del equipo de producir evidencia experimental, esta actividad evalúa la capacidad individual de interpretar esa evidencia y decidir con ella.

PE-U4 comparó cinco transformaciones (T1–T5) implementadas de forma equivalente en pandas y en PySpark sobre un conjunto sintético de 500,000 solicitudes de reserva y 40 laboratorios, midiendo cada transformación con `time.perf_counter()`, cinco repeticiones y mediana como estadístico central, tras descartar una ejecución de calentamiento. Los datos base de este informe proceden del repositorio grupal `pe-u4-spark-equipo-c`:

II. ANÁLISIS PROPIO DE LOS RESULTADOS DE SPEEDUP

II-A. Datos base

La Tabla I reproduce, con los valores propios tomados del commit declarado, la plantilla de datos base exigida. Todos los tiempos son medianas de cinco repeticiones; la plataforma de ejecución es un clúster Spark Standalone local (no Colab ni Databricks), con PySpark –versión fijada en `requirements.txt` del repositorio e instalada explícitamente en el entorno virtual antes de ejecutar la medición– y Python 3,12, idéntico al usado por el Equipo C en PE-U4.

Las columnas E de ambas tablas se calculan como la eficiencia del paralelismo,

$$E(N) = \frac{S(N)}{N}, \quad (1)$$

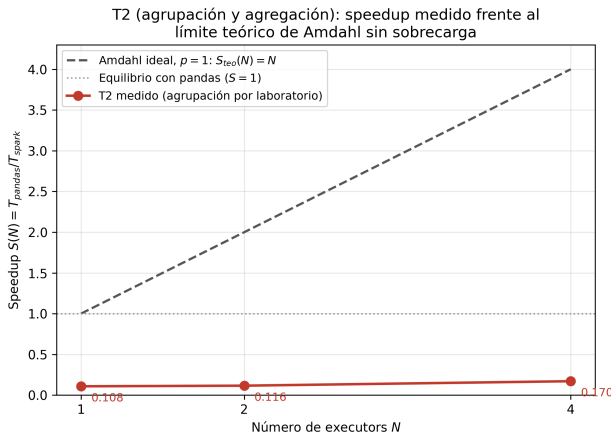
que en el caso de T2 cae de forma casi monotónica pese a que $S(N)$ crece levemente: al pasar de $N = 1$ a $N = 4$,

Tabla I
TIEMPOS MEDIANOS, SPEEDUP Y EFICIENCIA POR TRANSFORMACIÓN
(COMMIT A5FFCD9, $N = 4$ EXECUTORS SALVO DONDE SE INDICA).

Transf.	T_{pandas} (s)	T_{spark} (s)	S	$E(N=4)$
T1	0,2119	0,3457	0,6131	0,1533
T2 (foco)	0,1171	0,6875	0,1704	0,0426
T3	0,2889	0,8657	0,3337	0,0834
T4	2,2659	1,8100	1,2519	0,3130
T5	1,5252	1,3565	1,1244	0,2811

Tabla II
ESCALADO DE T2 (TRANSFORMACIÓN FOCO) CON $N = 1, 2, 4$
EXECUTORS.

N	$T_{\text{spark}, N}$ (s)	$S(N)$	$E(N)$
1	1,0850	0,1080	0,1080
2	1,0141	0,1155	0,0578
4	0,6875	0,1704	0,0426



según (1), la eficiencia se reduce de $E(1) = 0,1080$ a $E(4) = 0,0426$ (Tabla II), porque el denominador crece linealmente mientras el numerador apenas lo hace.

II-B. Contraste con la Ley de Amdahl

Con p la fracción paralelizable y N el número de unidades de procesamiento, la Ley de Amdahl establece

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (2)$$

En el caso límite $p = 1$ (paralelismo perfecto, sobrecarga nula), la ecuación (2) se reduce a $S_{\text{teo}}(N) = N$: la cota superior absoluta que cualquier implementación real puede aspirar a alcanzar. La Fig. 1 superpone esta cota sobre el speedup medido de T2 para $N = 1, 2, 4$.

La desviación relativa se calcula como $\Delta(N) = (S_{\text{med}}(N) - S_{\text{teo}}(N))/S_{\text{teo}}(N)$, expresada en porcentaje. Con $S_{\text{teo}}(N) = N$:

$$\Delta(1) = \frac{0,1080 - 1}{1} = -89,20 \%, \quad (3)$$

$$\Delta(2) = \frac{0,1155 - 2}{2} = -94,23 \%, \quad (4)$$

$$\Delta(4) = \frac{0,1704 - 4}{4} = -95,74 \%. \quad (5)$$

La desviación no solo es negativa sino creciente en magnitud con N : cuanto más recursos se añaden, más se aleja el sistema real de la cota ideal, porque el numerador (speedup medido) crece de forma casi plana mientras el denominador (cota teórica) crece linealmente.

II-C. Respuesta explícita: el caso $S < 1$

Los resultados de T2 **no coinciden** con la predicción de Amdahl bajo ninguna configuración medida: el speedup permanece por debajo de la unidad en $N = 1$, $N = 2$ y $N = 4$, es decir, la versión distribuida es siempre más lenta que la versión secuencial en pandas. La Ley de Amdahl, en su formulación clásica (2), no puede predecir este caso porque se deriva bajo la hipótesis de sobrecarga nula: el único costo que el modelo reconoce es el de la fracción estrictamente serial, y el único efecto de añadir procesadores es acelerar la fracción paralela. Un motor distribuido real, en cambio, introduce un costo de coordinación –arranque de sesión, planificación del DAG, serialización y shuffle– que el modelo simplemente no representa [2]. Cuando ese costo de coordinación por sí solo supera el tiempo total de la alternativa secuencial, como ocurre aquí, ningún valor de $p < 1$ en la ecuación (2) puede producir $S < 1$: el modelo está, por construcción, fuera de su dominio de aplicación válido para este caso. Esta observación es exactamente la formalizada por Hill y Marty al argumentar que, en arquitecturas multinúcleo y distribuidas reales, la ley de Amdahl clásica sistemáticamente sobreestima el rendimiento alcanzable porque ignora los costos de sincronización y comunicación que crecen con N [2].

III. DIAGNÓSTICO DE LA FRACCIÓN NO ESCALABLE

III-A. Métrica de Karp-Flatt y su dominio de validez

La métrica de Karp-Flatt despeja, a partir del speedup observado S con N unidades, la fracción serial experimental

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1, \quad (6)$$

propuesta originalmente como diagnóstico de la porción no paralelizable de un cómputo real [3]. Sustituyendo los valores de la Tabla II:

$$e(2) = \frac{\frac{1}{0,1155} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{8,6579 - 0,5}{0,5} = 16,316, \quad (7)$$

$$e(4) = \frac{\frac{1}{0,1704} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{5,8697 - 0,25}{0,75} = 7,492. \quad (8)$$

Ambos valores son mayores que 1, lo cual es matemáticamente imposible para una fracción serial: por definición, $e \in [0, 1]$. Este no es un error de cálculo, sino la consecuencia directa de que la ecuación (6) es algebraicamente equivalente a despejar p de (2), y por tanto hereda la misma hipótesis de sobrecarga nula. Cuando $S < 1$, como ocurre en las tres configuraciones de T2, la fórmula produce un valor fuera de su dominio válido: el resultado debe interpretarse como una señal de que el modelo de dos términos (fracción serial fija más fracción paralela perfectamente escalable) es inadecuado para describir esta transformación, no como una fracción serial real de 1631 % o 749 %.

III-B. Lectura de la tendencia

Aunque los valores individuales de e carecen de significado como fracción, su **tendencia** sí es informativa: e disminuye de 16,316 en $N = 2$ a 7,492 en $N = 4$, es decir, a la inversa que si la sobrecarga creciera con el número de executors (lo que se manifestaría como e creciente, según el criterio de interpretación estándar de la métrica). La disminución observada es consistente con una interpretación específica: el costo dominante en T2 es un costo **fijo** por ejecución (arranque de la sesión Spark, planificación del plan físico, apertura de conexiones entre executors), no un costo que crezca con N ; al aumentar N , ese costo fijo se diluye levemente porque el shuffle en sí se paraleliza entre más executors, lo que hace que $S(N)$ crezca muy poco (de 0,108 a 0,170) y, en consecuencia, que el valor calculado de e —que depende inversamente de S — disminuya. La lectura correcta no es “el algoritmo escala mal” ni “la sobrecarga crece con N ”, sino que **el costo de coordinación fijo del motor es, por sí solo, mayor que todo el presupuesto de tiempo de la alternativa secuencial**, independientemente de N .

III-C. Atribución a mecanismos concretos

Se identifican tres mecanismos concretos, no genéricos, que explican la sobrecarga de T2, cada uno vinculado a evidencia propia verificable en el repositorio (no a una Spark UI capturada específicamente para T2, que no forma parte de la evidencia de PE-U4, sino a las salidas de datos y a las mediciones ya reportadas):

1. **Shuffle desproporcionado al resultado útil.** T2 agrupa 500,000 filas de solicitudes en solo 40 grupos de salida (uno por laboratorio). El motor debe redistribuir por red la totalidad de las filas de entrada antes de poder aplicar la agregación, aun cuando el resultado final es minúsculo. *Evidencia propia:* la salida persistida `data/spark/T2_agrupacion/` del repositorio de PE-U4 se escribe en una única partición de archivo, frente a las dos particiones que exhiben las salidas de T1, T3 y T4; el resultado final de T2 es tan pequeño que ni siquiera justifica más de un archivo de salida, lo que contrasta directamente con el volumen que debió moverse por la red durante el shuffle previo. Yu et al. documentan que la operación de reducción/agregación en Spark es el cuello de botella de escalabilidad menos favorable del motor cuando el volumen de datos movidos por el shuffle no guarda proporción con el trabajo útil de reducción posterior [4].
2. **Arranque y planificación de sesión.** Cada configuración de N se midió con una sesión Spark Standalone independiente, según el protocolo de PE-U4; el establecimiento de la conexión con el clúster, el registro de executors y la construcción del plan físico por el optimizador Catalyst introducen un costo fijo por ejecución que no existe en pandas. *Evidencia propia:* con un único executor ($N = 1$, sin ningún costo de coordinación entre executors posible), T2 en PySpark ya mide 1,0850 s frente a 0,1171 s de pandas (Tabla II), una razón de 9,27×; puesto que con un solo executor no hay comunicación inter-nodo que explicar, esa brecha completa es atribuible al costo fijo de sesión y planificación, no a la coordinación entre múltiples executors [5].
3. **Ausencia de agregación parcial efectiva a esta cardinalidad.** *Evidencia propia:* la verificación de equivalencia del notebook de PE-U4 confirma que la salida de T2 tiene exactamente 40 filas, idénticas entre pandas y PySpark. Con solo 40 claves de salida, la etapa de reducción posterior al shuffle es casi instantánea; todo el tiempo de la transformación distribuida se concentra en mover datos, no en calcularlos, de modo que añadir executors divide el trabajo de shuffle entre más unidades sin reducir el volumen total transferido por la red.

IV. PROPUESTA DE INTEGRACIÓN AL PFC

IV-A. Caso de uso

En el SCLI, el proceso análogo a T2 es la generación periódica de un **reporte de demanda por laboratorio**: agrupar las solicitudes de reserva registradas en un período, contar cuántas corresponden a cada laboratorio y calcular la ocupación promedio y máxima. La salida de este reporte alimenta una decisión operativa concreta: la **reprogramación de mantenimiento preventivo y la reasignación**

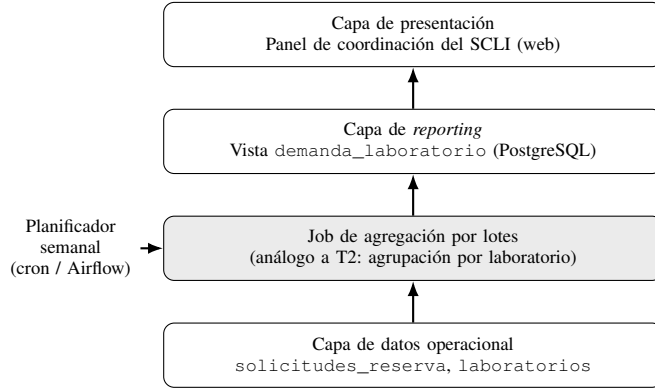


Figura 2. Punto de inserción del procesamiento por lotes en la arquitectura en capas del SCLI. El job se dispara semanalmente, lee de la capa operacional y escribe exclusivamente en una capa de *reporting* aislada, sin afectar la disponibilidad transaccional. Diagrama de elaboración propia (TikZ).

de cupos de horario hacia los laboratorios con mayor demanda sostenida, decisión que hoy en el SCLI se toma sin ese insumo cuantitativo.

IV-B. Periodicidad y volumen

A diferencia de una consulta transaccional (por ejemplo, verificar disponibilidad al momento de reservar), este reporte no requiere latencia baja: tiene sentido ejecutarlo por lotes, con periodicidad **semanal** (coincidente con el ciclo de planificación académica) o, como mínimo, mensual. El volumen actual estimado de solicitudes por semestre en un despliegue real de la Facultad es de algunos miles a pocas decenas de miles de registros –muy por debajo del umbral de rentabilidad calculado en la Sección V–, aunque el volumen acumulado histórico crece con cada período académico.

IV-C. Arquitectura de integración

La Fig. 2 muestra el punto exacto de inserción en la arquitectura en capas del SCLI. El procesamiento se ejecuta **por lotes**, no en flujo: un planificador (por ejemplo, un *cron job* o un DAG de Airflow) dispara semanalmente un job de agregación que lee de la capa de datos operacional (la tabla *solicitudes_reserva* de PostgreSQL), calcula la agregación por laboratorio y escribe el resultado en una capa de *reporting* separada (una tabla o vista materializada *demanda_laboratorio*), que a su vez alimenta el panel de coordinación consumido por la capa de presentación. El job de agregación nunca escribe sobre la base operacional, de modo que un fallo o una demora en el reporte no compromete la disponibilidad del sistema transaccional de reservas.

V. DIMENSIONAMIENTO Y VIABILIDAD

V-A. Umbral de rentabilidad

Modelando el tiempo secuencial como $T_{\text{sec}}(n) = \alpha n$ y el distribuido como $T_{\text{dist}}(n) = \beta + \gamma n/N$, donde β agrupa

Tabla III
UMBRAL DE RENTABILIDAD n^* EXTRAPOLADO PARA $N > 4$ (FUERA DEL RANGO MEDIDO; EL MODELO SOLO FUE AJUSTADO CON $N = 1, 2, 4$).

N (executors)	n^* (registros)
5	14 665 148
8	5 638 763
16	3 727 083

la sobrecarga fija y n es el número de registros, el motor distribuido resulta ventajoso cuando $T_{\text{dist}}(n) < T_{\text{sec}}(n)$, es decir:

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (9)$$

Los parámetros se estimaron por regresión lineal de mínimos cuadrados sobre las tres mediciones propias de T2 ($N = 1, 2, 4$, con $n = 500,000$ fijo), tomando $x = 1/N$ como variable independiente: $\alpha = T_{\text{pandas}}/n = 2,3427 \times 10^{-7}$ s por registro; $\beta = 0,6521$ s (ordenada al origen del ajuste); y $\gamma = 9,4903 \times 10^{-7}$ s por registro (pendiente del ajuste dividida entre n).

Sustituyendo en la condición de validez de (9) con $N = 4$, la configuración usada en PE-U4:

$$\alpha - \frac{\gamma}{4} = 2,3427 \times 10^{-7} - 2,3726 \times 10^{-7} = -2,99 \times 10^{-9} < 0. \quad (10)$$

La condición **no se cumple**: con $N = 4$ executors, el costo marginal por registro del motor distribuido es, por muy poco, mayor que el de pandas. Bajo este modelo lineal, **no existe ningún volumen finito** de datos que rentabilice Spark para T2 con esa configuración de clúster –el umbral n^* diverge–. Despejando el N mínimo para el cual la condición se satisface, $N^* = \gamma/\alpha \approx 4,05$: el punto de equilibrio está justo por encima de los cuatro executors usados en la práctica. La Tabla III extrapola n^* para configuraciones mayores, fuera del rango medido, declarando explícitamente esa extrapolación como limitación del ajuste.

V-B. Modelo de servicio en la nube

Dado que el volumen proyectado del SCLI (Sección IV) está muy por debajo de cualquier n^* de la Tabla III, mantener un clúster Spark permanente sería un sobre-dimensionamiento. Se recomienda un modelo **PaaS con aprovisionamiento elástico bajo demanda** –equivalente a un servicio gestionado tipo EMR/HDInsight/Datapro– que solo se instancia si, en el futuro, el volumen supera el umbral calculado; esto aprovecha la característica de elasticidad rápida del cómputo en la nube [6] sin incurrir en el costo fijo de un clúster ocioso la mayor parte del tiempo. Mientras el volumen se mantenga por debajo del umbral, el job de agregación de la Fig. 2 debe ejecutarse directamente sobre la base de datos operacional (Sección VI).

V-C. Recursos y riesgos

Los recursos requeridos bajo el modelo recomendado son mínimos: el mismo servidor que aloja PostgreSQL puede ejecutar la consulta de agregación periódica sin infraestructura adicional. El principal riesgo identificado es de **validez del modelo**: α , β y γ se estimaron sobre un conjunto sintético y con solo tres puntos de medición; el umbral debe recalcularse cuando existan mediciones sobre datos reales de producción del SCLI, y la extrapolación de la Tabla III para $N > 4$ debe tratarse como orientativa hasta que se mida directamente.

VI. POSTURA CRÍTICA Y ALTERNATIVAS

Con la evidencia de las secciones anteriores, la postura de este informe es que **Spark no es la opción correcta** para el reporte de demanda por laboratorio del SCLI en su estado actual: el speedup medido es inferior a 1 en toda la escala evaluada, la condición de rentabilidad de la ecuación (9) no se satisface con la configuración de clúster disponible, y el volumen proyectado del sistema está muy por debajo de cualquier umbral extrapolado razonable. Esta conclusión es consistente con el argumento de McSherry et al., quienes muestran que numerosos sistemas de macrodatos publicados requieren cientos de núcleos antes de superar el desempeño de una implementación de un solo hilo bien escrita, y que la escalabilidad por sí sola –sin comparar contra ese punto de referencia– es una métrica engañosa [7].

Se evaluaron y descartaron dos alternativas distribuidas antes de llegar a la recomendación final:

- **Dask.** Ofrece una API similar a pandas con ejecución distribuida y podría reducir la curva de aprendizaje frente a PySpark. Se descarta porque su operación `groupby` sobre claves de baja cardinalidad exhibe el mismo patrón de shuffle completo que Spark: el problema de fondo (movimiento de datos desproporcionado al resultado) no es específico del motor, sino de la operación, y añadir un planificador distribuido introduciría complejidad operativa sin resolver la causa raíz.
- **Almacén de datos analítico gestionado en la nube** (por ejemplo, un servicio tipo BigQuery o Redshift). Se descarta por desproporción: el costo recurrente y el bloqueo de proveedor no se justifican para un reporte semanal sobre un volumen de datos que cabe cómodamente en una única tabla indexada de PostgreSQL.

La alternativa recomendada –ya integrada en la Fig. 2– es ejecutar la agregación directamente como una consulta `GROUP BY` indexada sobre la base operacional del SCLI, revalidando periódicamente el umbral n^* a medida que el sistema acumula historial. Esta postura no es definitiva: si el SCLI evolucionara hacia un volumen de solicitudes multi-institucional, o si el mecanismo de agregación cambiara para reducir el shuffle (por ejemplo, mediante

agregación parcial en el lado del mapa), la conclusión debería revisarse.

VII. CONCLUSIONES

1. El speedup de T2 resultó inferior a la unidad en las tres configuraciones medidas (0,108, 0,116 y 0,170 para $N = 1, 2, 4$), un caso que la Ley de Amdahl clásica no puede predecir por estar fuera de su hipótesis de sobrecarga nula [1], [2]; esta evidencia muestra en la práctica que un motor distribuido introduce un costo de coordinación que ninguna cantidad de paralelismo algorítmico compensa cuando el resultado útil es pequeño frente al volumen de entrada.
2. La métrica de Karp-Flatt, calculada sobre los datos propios de T2, arrojó valores fuera de su dominio válido ($e(2) = 16,32$, $e(4) = 7,49$), lo que –lejos de ser un error– constituye evidencia diagnóstica de que el modelo de dos términos que sustenta tanto a Amdahl como a Karp-Flatt es inadecuado para transformaciones dominadas por un costo fijo de coordinación, como el shuffle de baja cardinalidad de T2 [3], [4].
3. El ajuste del modelo de umbral de rentabilidad sobre las mediciones propias mostró que, con la configuración de clúster de PE-U4 ($N = 4$), ningún volumen de datos hace rentable a Spark para esta operación, y que se requieren al menos cinco executors para que el umbral vuelva a ser finito; esta cifra, aplicada al SCLI, respalda una decisión de arquitectura concreta: mantener la agregación de demanda por laboratorio en la base de datos operacional en lugar de introducir un motor distribuido de forma prematura.

VIII. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

Se emplearon dos herramientas de inteligencia artificial generativa como apoyo puntual en la elaboración de este informe, cuyo contenido técnico, análisis e interpretación son de autoría y responsabilidad del estudiante. **Claude (Anthropic)** se usó para: corregir ortografía, tildes y redacción en español técnico; ordenar la estructura del documento conforme a las diez secciones exigidas por la guía; verificar de forma independiente los cálculos numéricos ya resueltos por el estudiante (sustitución en las ecuaciones de Amdahl, Karp-Flatt, eficiencia y umbral de rentabilidad); resolver dudas puntuales sobre temas previamente desconocidos para el estudiante, en particular la sintaxis de $\text{\LaTeX}$ para entornos de ecuación numerados, tablas `booktabs` y el paquete `siunitx`, así como la construcción del diagrama en TikZ de la Fig. 2; y ayudar a localizar y verificar el DOI de las referencias bibliográficas citadas. **Codex (OpenAI)** se usó únicamente para ejecutar, sobre el clúster Standalone propio, el protocolo de medición de T2 con $N = 1$ y $N = 2$ executors ya definido y utilizado por el equipo para las demás configuraciones, sin modificar el

protocolo ni el código de las transformaciones existentes. Se declara además que la medición de T2 con $N = 1$ y $N = 2$ no existía en el commit original de PE-U4 –la guía grupal solo exigía ese escalado para T3– y fue generada expresamente para este informe individual, incorporándose al repositorio grupal en el commit bbe19c8 para preservar la trazabilidad exigida.

REFERENCIAS

- [1] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.
- [2] M. D. Hill y M. R. Marty, «Amdahl's Law in the Multicore Era,» *Computer*, vol. 41 n.º 7, págs. 33-38, 2008. DOI: 10.1109/MC.2008.209.
- [3] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33 n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [4] B. Yu, H. Cao, T. Shan, H. Wang, X. Tang y W. Chen, «Sparker: Efficient Reduction for More Scalable Machine Learning with Spark,» en *Proceedings of the 50th International Conference on Parallel Processing (ICPP '21)*, Lemont, IL, USA: Association for Computing Machinery, 2021, págs. 1-11. DOI: 10.1145/3472456.3472499.
- [5] M. Zaharia et al., «Apache Spark: A Unified Engine for Big Data Processing,» *Communications of the ACM*, vol. 59 n.º 11, págs. 56-65, 2016. DOI: 10.1145/2934664.
- [6] P. Mell y T. Grance, «The NIST Definition of Cloud Computing,» National Institute of Standards y Technology, Gaithersburg, MD, USA, inf. téc. Special Publication 800-145, 2011. DOI: 10.6028/NIST.SP.800-145.
- [7] F. McSherry, M. Isard y D. G. Murray, «Scalability! But at What COST?» En *Proceedings of the 15th USENIX Conference on Hot Topics in Operating Systems (HotOS'15)*, Berkeley, CA, USA: USENIX Association, 2015. DOI: 10.5555/2831090.2831104.